

Abracadabracadoo Message Flow (Combined Precise + Descriptive)

1 Key Establishment

Alice and Bob share `K_msg` through a secure handshake so only they can read the message. Separately, Alice and the server share `K_proof` for proof mediation.

Action:

This ensures that message encryption remains private between Alice and Bob while allowing the server to facilitate verifiable delivery without seeing the message content.

2 Alice Prepares Message

```
M = "Penguins are really cool"
```

Alice writes the message she wants to send securely and verifiably to Bob.

3 Alice Generates Proof Token

```
T = timestamp  
P = HMAC(K_proof, M || T)  
H_P = SHA256(P)
```

Alice creates a proof token tied to the message and timestamp, generating a hash for unique fingerprinting.

4 Alice Encrypts Message

```
nonce = H_P[0..11]  
EM = AEAD_Enc(K_msg, nonce, M)
```

Using the hashed proof token, Alice derives a nonce to encrypt the message securely and uniquely.

5 Alice Encrypts Proof

```
H_EM = SHA256(EM)
nonce = H_EM
EP = AEAD_Enc(K_proof, nonce, P)
```

Alice hashes the encrypted message and uses it as the nonce to encrypt the proof, producing **EP**.

6 Alice Sends to Server

Alice sends:

```
(msgID, EM, EP)
```

Server logs:

```
(msgID, H_EM, T1)
```

Alice sends the encrypted message and proof to the server, which logs a hash and timestamp while preserving content privacy.

7 Bob Receives Message

Bob receives:

```
(EM, EP)
```

Computes:

```
H_EM' = SHA256(EM)
```

Sends:

```
(msgID, H_EM')
```

Bob receives the encrypted materials, computes the hash, and sends it back to the server for confirmation.

8 Server Verifies and Releases Proof

Confirms:

```
H_EM' == H_EM
```

Decrypts:

```
P = AEAD_Dec(K_proof, nonce=H_EM, EP)
```

Logs:

```
(msgID, P, T2)
```

Sends:

```
P
```

The server verifies Bob's confirmation, decrypts the proof, logs it, and sends it to Bob.

9 Bob Decrypts Message

```
H_P = SHA256(P)
nonce = H_P[0..11]
M = AEAD_Dec(K_msg, nonce, EM)
```

Bob uses the proof to derive the nonce and decrypts the message, revealing:

```
M = "Penguins are really cool"
```

10 Optional Verification

Any party may verify: - Server logs and hashes. - EP decrypts to P using K_proof. - EM decrypts to M using K_msg with nonce SHA256(P)[0..11].

This ensures Bob received and decrypted the exact message, with verifiable privacy and sovereignty throughout the process.

This combined structure preserves technical precision while offering clear context for each step, making it an ideal reference for technical onboarding and ecosystem clarity.